

SMARTPARK KE

Intelligent Automated Parking Management System Technical Blueprint, Polyglot Architecture & Data Structures Report

Target Jurisdiction:	Republic of Kenya (Safaricom Lipa na M-Pesa Integration)
Polyglot Services:	Python (Web Gateway) • C++17 (Slot Engine) • Java 17 (Billing)
Persistence:	Dynamic PostgreSQL / MySQL Schema with B-Tree Indices
Tariff Model:	Dynamic Data-Driven Table (0 to 30m Free, 50, 100, 300, 500 Kshs)
Author / System:	Google DeepMind Advanced Agentic Coding Pair (Antigravity)

1. Problem Analysis & Client Terms of Reference

Commercial and municipal parking operations across urban centers in Kenya encounter major operational bottlenecks: manual gate ticketing, slow cash and card reconciliation, congestion at entrance gates due to absent real-time bay guidance, and revenue leakage. SmartPark KE was engineered to provide a fully automated, modern web-based parking management system conforming strictly to the client terms of reference below:

- **Pre-Entry Visual Display:** Real-time visual color-coded layout of bays before entry.
- **Vehicle Registration:** Automatic timestamped check-in recording plate number and vehicle category.
- **Automated Exit Calculation:** Automatic elapsed duration computation and tariff calculation on exit.
- **Automated Barrier Actuator:** Barrier automatically lifts upon verified fee payment.
- **Mobile Money Integration:** Safaricom Daraja Lipa na M-Pesa STK Push direct to customer phone.
- **Nearest Slot Allocation:** Min-Heap and Dijkstra graph routing allocating closest available bay.
- **Real-Time Stay Tracking:** Continuous in-memory duration tracking and overstay detection.
- **Fixed Tariff Schedule:** 0-30 min (Free), 2h (50 Kshs), 4h (100 Kshs), 6h (300 Kshs), >6h (500 Kshs).

2. Polyglot Microservices Architecture

The architecture decomposes system responsibilities across three languages suited to their specialized operational profiles:

- **C++17 Engine (Port 8081):** Manages the free-slot Min-Heap, multi-floor driving Graph Dijkstra shortest-paths, physical row Doubly Linked Lists, and the barrier Finite State Machine (FSM) with a LIFO audit stack.
- **Java 17 Spring Boot (Port 8082):** Manages dynamic tariff array scans, Safaricom Daraja STK Push requests, asynchronous callback webhook processing via a FIFO Queue, and SMS receipt dispatch.
- **Python 3.11 Gateway (Port 5000):** Serves HTML5/JS clients, handles active session hash maps, overstay min-heap peek checks, relational persistence via SQLAlchemy, and pushes live slot diffs over WebSockets.

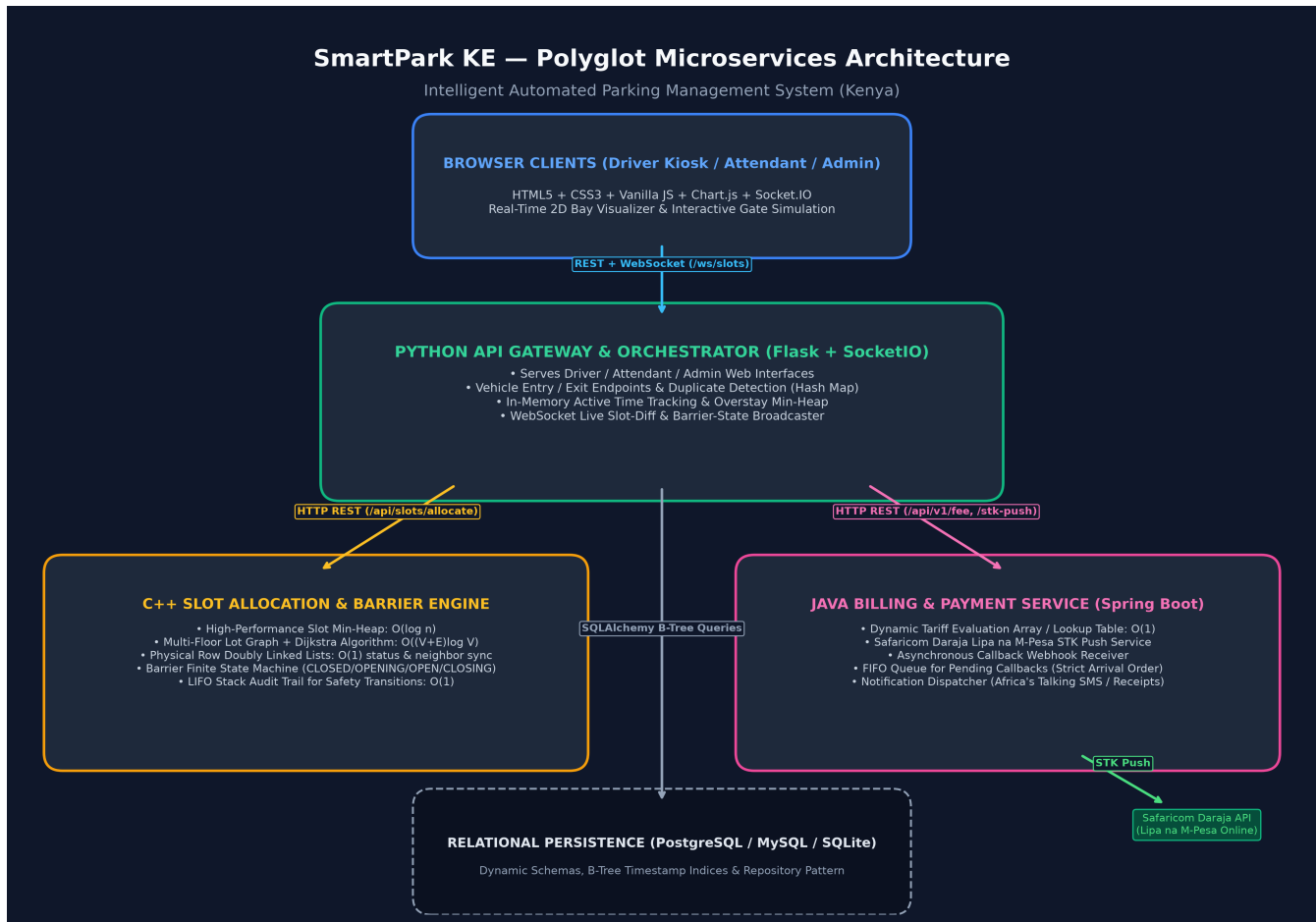


Figure 1: Polyglot Microservices Architecture and Inter-Service REST/WebSocket Communication

3. Data Structures Summary & Algorithmic Justifications

Every data structure in SmartPark KE was selected based on Big-O algorithmic guarantees to ensure maximum responsiveness and zero bottlenecks:

Data Structure	Module	Time Complexity	Engineering Justification
Hash Map	1, 4, 10	O(1) Get/Put	Instant duplicate check on entry, active stay lookup, and auth token validation.
Min-Heap	2, 4	O(log n) Push/Pop O(1) Peek	Allocates closest bay by distance in O(log n); peeks oldest vehicle for overstay alerts in O(1).
Weighted Graph + Dijkstra	2	O((V+E)log V)	Computes exact multi-floor driving distances across ramps, aisles, and turning radiuses.
Doubly Linked List	2	O(1) Insert/Update	Re-inserts freed bay next to physical neighbors in O(1), keeping visual ordering pristine.
2D Matrix Array	3	O(1) Cell Access	Directly mirrors physical bays (rows × cols) for real-time browser rendering.
FIFO Queue	6, 11	O(1) Enqueue/Deq	Preserves strict arrival order of asynchronous Daraja payment callbacks and SMS jobs.
LIFO Stack	7	O(1) Push/Peek	Maintains chronological audit trail of gate transitions; top yields most recent event.
Lookup Table	5	O(1) Tier Scan	Scans small dynamic tariff list (5 tiers) linearly with zero overhead.
B-Tree (DB Index)	8, 9	O(log N) Range	Relational database index enabling ultra-fast range queries on entry and exit timestamps.

4. Dynamic Relational Database Design

The relational database schema is fully dynamic and data-driven. Tariff schedules, multi-level zones, and bay geometries are stored in configurable tables rather than hardcoded:

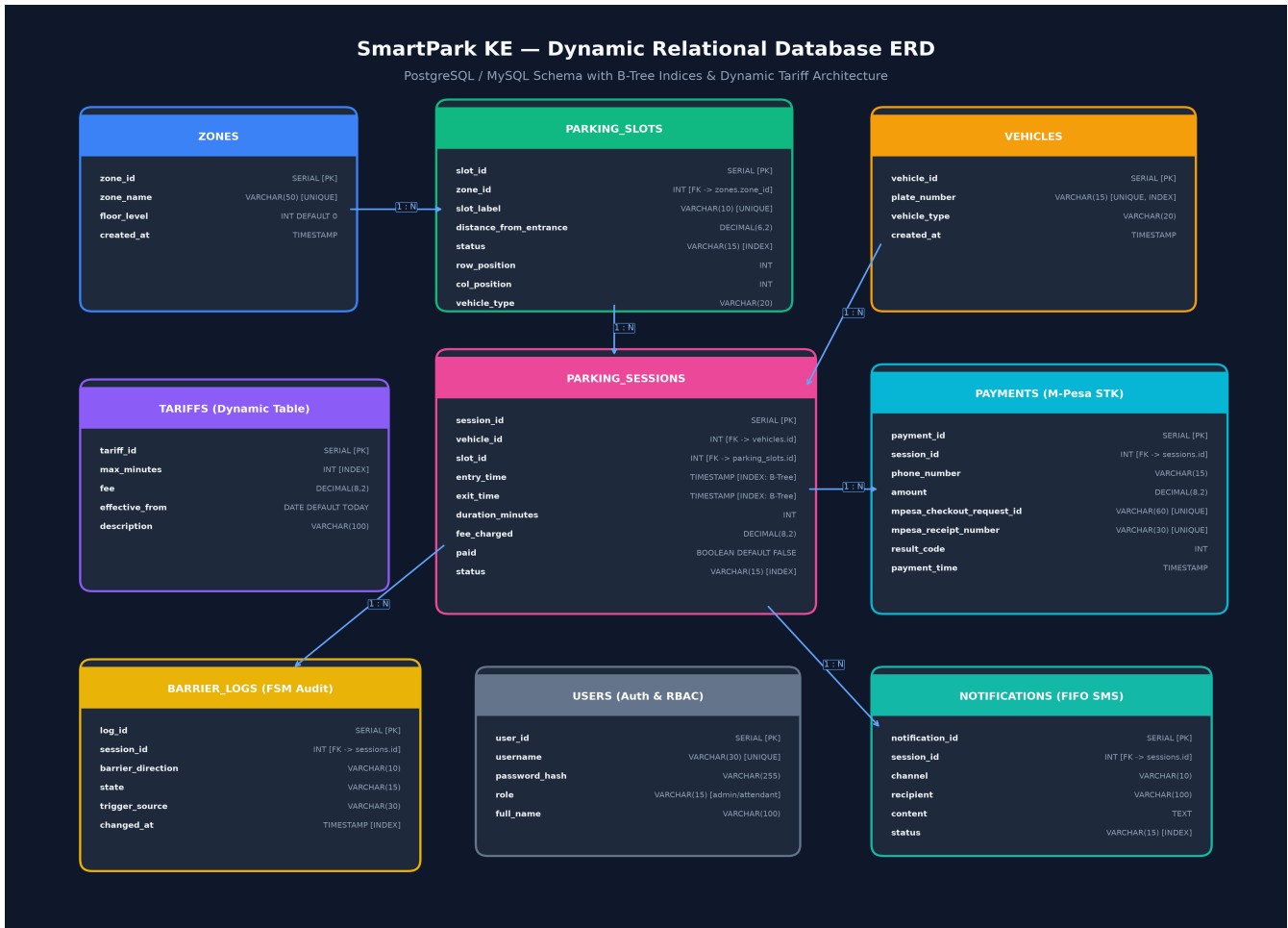


Figure 2: Dynamic Relational Database ERD with B-Tree Indices and Foreign Key Topology

5. Algorithmic Pseudocode Specifications (All 11 Modules)

Module 1: Vehicle Entry & Registration

```
// O(1) duplicate check + session creation
function RegisterArrival(plateNumber, vehicleType):
    if ActiveSessionsHashMap.contains(plateNumber):
        throw DuplicateEntryException("Vehicle already inside lot")
    slot = SlotEngineClient.allocateNearestSlot()
    session = DB.createSession(plateNumber, slot.id, CurrentTime())
    ActiveSessionsHashMap.put(plateNumber, session)
    TimeTrackerMinHeap.push(session.entryTime, session)
    WebSocket.broadcastSlotDiff(slot.id, "OCCUPIED", plateNumber)
    return session
```

Module 2: Nearest Slot Allocation (C++ Min-Heap + Dijkstra)

```
// O(log n) slot extraction from MinHeap
function ExtractNearestSlot():
    if freeSlotsHeap.isEmpty():
        return NULL
    nearest = freeSlotsHeap.extractMin() // O(log n)
    rowList = zoneRowDoublyLinkedLists[nearest.zoneId]
    rowList.updateStatus(nearest.slotId, "OCCUPIED") // O(1)
    return nearest

// Dijkstra shortest driving path calculation across multi-floor lot
function DijkstraShortestPaths(entranceNodeId):
    dist[v] = INFINITY for all v; dist[entranceNodeId] = 0
    pq.push((0, entranceNodeId))
    while not pq.isEmpty():
        (currDist, u) = pq.pop()
        for edge in adjList[u]:
            if dist[u] + edge.weight < dist[edge.v]:
                dist[edge.v] = dist[u] + edge.weight
                pred[edge.v] = u
                pq.push((dist[edge.v], edge.v))
    return dist, pred
```

Module 5: Fee Calculation (Java Dynamic Tariff Table)

```
// O(1) linear scan through dynamic tariff array
function CalculateFee(durationMinutes):
    duration = max(0, durationMinutes)
    for tier in dynamicTariffTable:
        if duration <= tier.maxMinutes:
            return tier.fee
    return 500.00 // Default over-6-hour rate
```

Module 6: Safaricom Daraja Lipa na M-Pesa STK Push

```
// Initiates STK Push & enqueues asynchronous callback in FIFO order
function ProcessStkPush(sessionId, phone, amount):
    normPhone = FormatKenyanNumber(phone) // 2547XXXXXXX
    timestamp = CurrentDateTime("yyyyMMddHHmmss")
    password = Base64(ShortCode + PassKey + timestamp)
    checkoutId = DarajaAPI.sendSTKPrompt(normPhone, amount, password, timestamp)
    PendingPaymentsMap.put(checkoutId, {sessionId, amount, normPhone})
    return checkoutId

function OnDarajaWebhook(callbackPayload):
    FifoCallbackQueue.offer(callbackPayload) // FIFO Queue
    while not FifoCallbackQueue.isEmpty():
        event = FifoCallbackQueue.poll()
        if event.resultCode == 0:
            ConfirmPaymentAndOpenBarrier(event.checkoutRequestId)
```

Module 7: Barrier Actuator State Machine (LIFO Audit Stack)

```
// FSM: CLOSED -> OPENING -> OPEN -> CLOSING -> CLOSED
function TriggerBarrierOpen(sessionId):
  if currentState == CLOSED or currentState == CLOSING:
    currentState = OPENING
    log = {from: "CLOSED", to: "OPENING", trigger: "PAYMENT_CONFIRMED", sessionId}
    AuditStack.push(log) // 0(1) LIFO Stack
    ActuatorHardware.raiseBoomArm()
    currentState = OPEN
    return true
  return false
```